

Graphics Processing Units (GPUs)

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

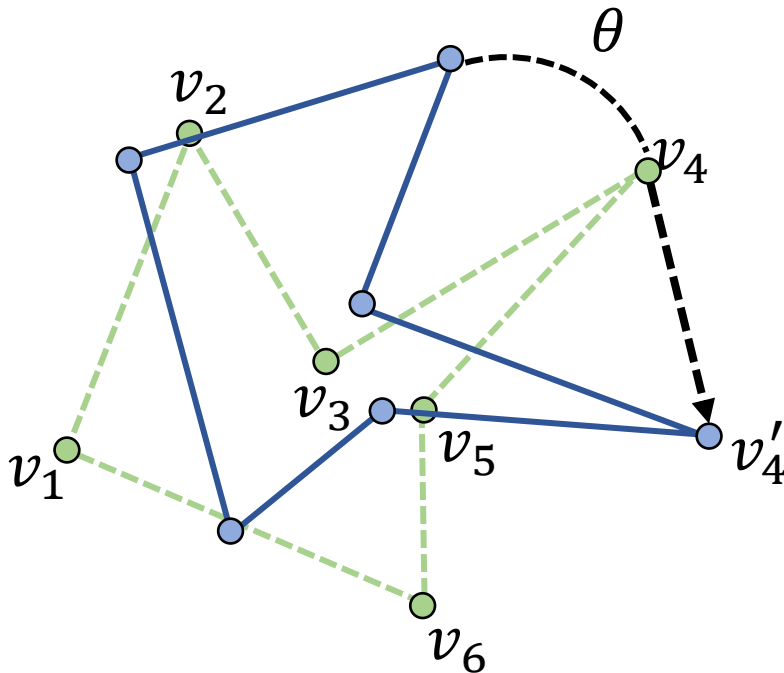
Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

Vector Graphics

- *polygon*: a series of line segments forming a closed planar shape
- *vertex*: a point joining two line segments
- *rotation*: 2D or 3D linear operation applied to a series of vertices



$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

$$v'_1 = Rv_1$$

$$v'_2 = Rv_2$$

$$v'_3 = Rv_3$$

$$v'_4 = Rv_4$$

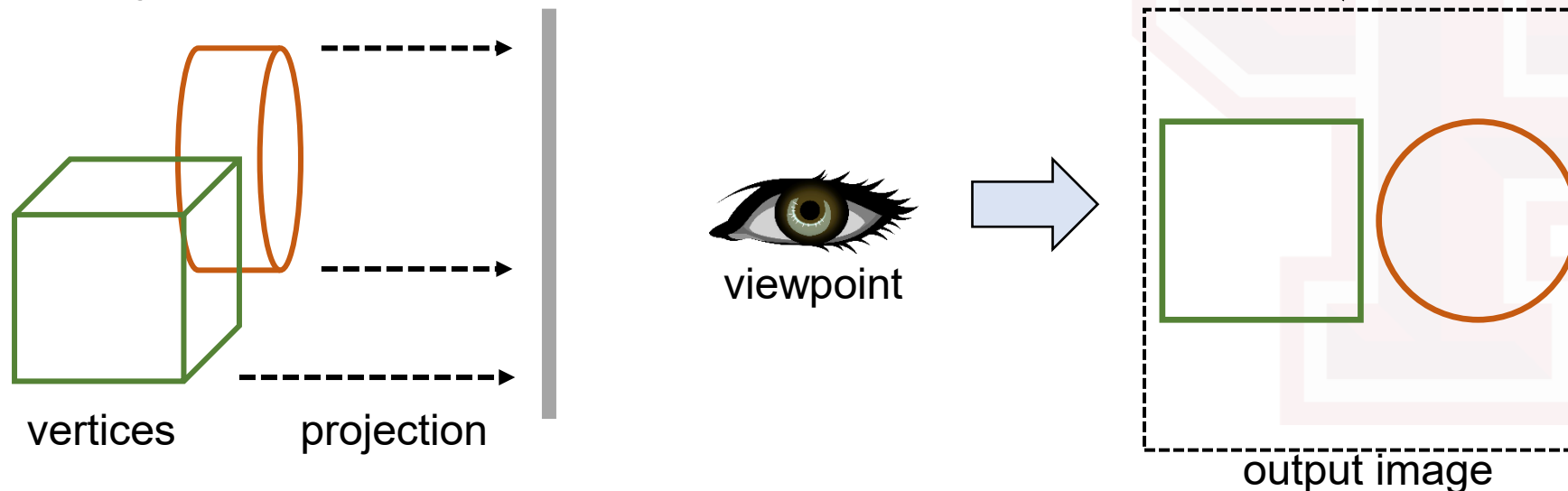
$$v'_5 = Rv_5$$

$$v'_6 = Rv_6$$

$$= \begin{bmatrix} x'_5 \\ y'_5 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_5 \\ y_5 \end{bmatrix}$$

Rendering a Virtual World

- 1) Describe the world as a series of vertices connected by lines
- 2) Create a virtual camera
- 3) Rotate all of the vertices to line the camera up with its position in the virtual world
- 4) Apply a projection to render the lines to a 2D plane (the screen)



Projections

- *orthogonal projection*: directly eliminate the z-axis component after rotating to the camera

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

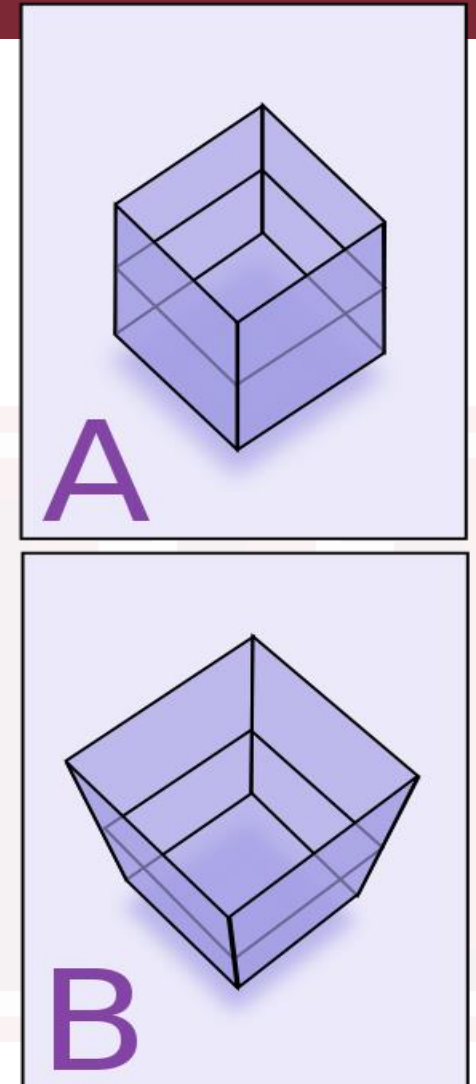
$$\begin{aligned} \dot{v}_1 &= Pv'_1 \\ \dot{v}_2 &= Pv'_2 \\ \dot{v}_3 &= Pv'_3 \\ &\dots \\ \dot{v}_{n-1} &= Pv'_{n-1} \\ \dot{v}_n &= Pv'_n \end{aligned}$$

- *perspective projection*: scale the (x, y) coordinates as a function of z (distance from the camera)
 - points must be represented as a 4D vector

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & D & 0 \end{bmatrix}$$

$$\dot{v}_n = \frac{Pv'_n}{w'_n}$$

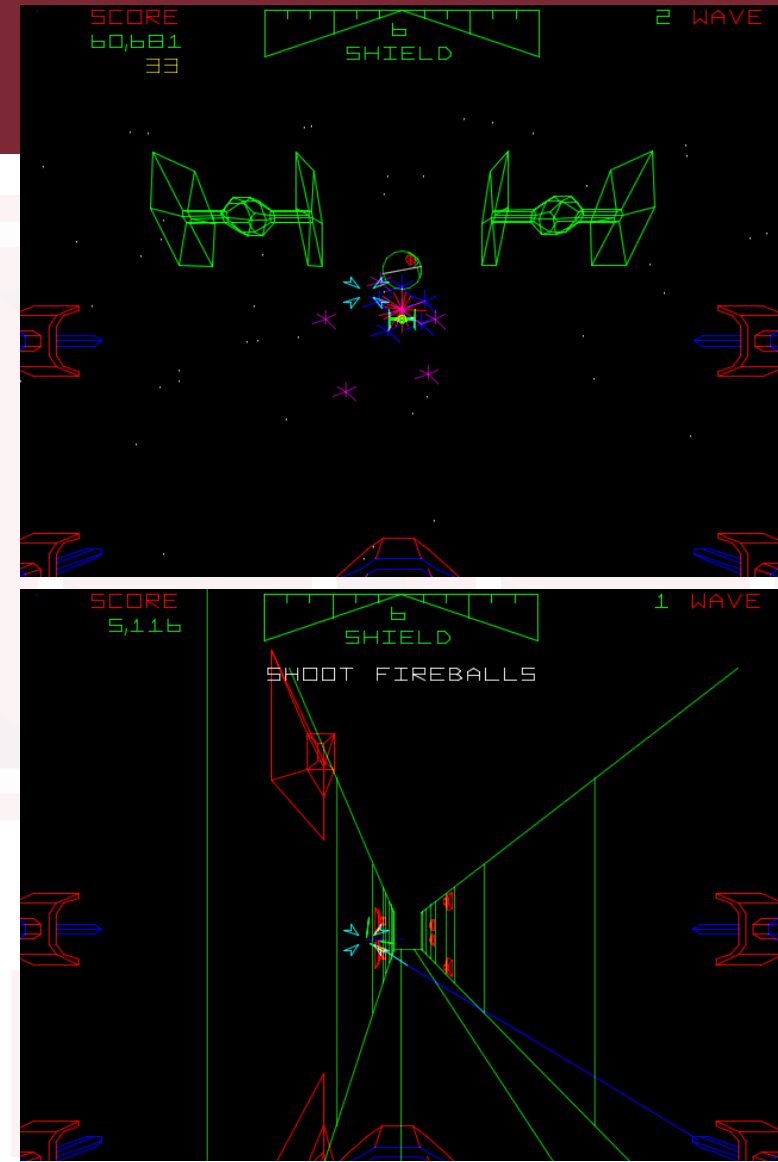
$$\text{where } v'_n = \begin{bmatrix} x'_n \\ y'_n \\ z'_n \\ w'_n \end{bmatrix}$$



Vector Graphics (1980s)

```
//calculate the view matrix R
//based on camera position
//calculate the projection matrix P
v = vertex[2]
for each polygon p
  for each vertex i in p
     $\dot{v}[1] = P * R * v_{pi}$ 
    if  $i \neq 0$ 
      draw_line( $\dot{v}[0]$ ,  $\dot{v}[1]$ )
     $\dot{v}[0] = \dot{v}[1]$ 
```

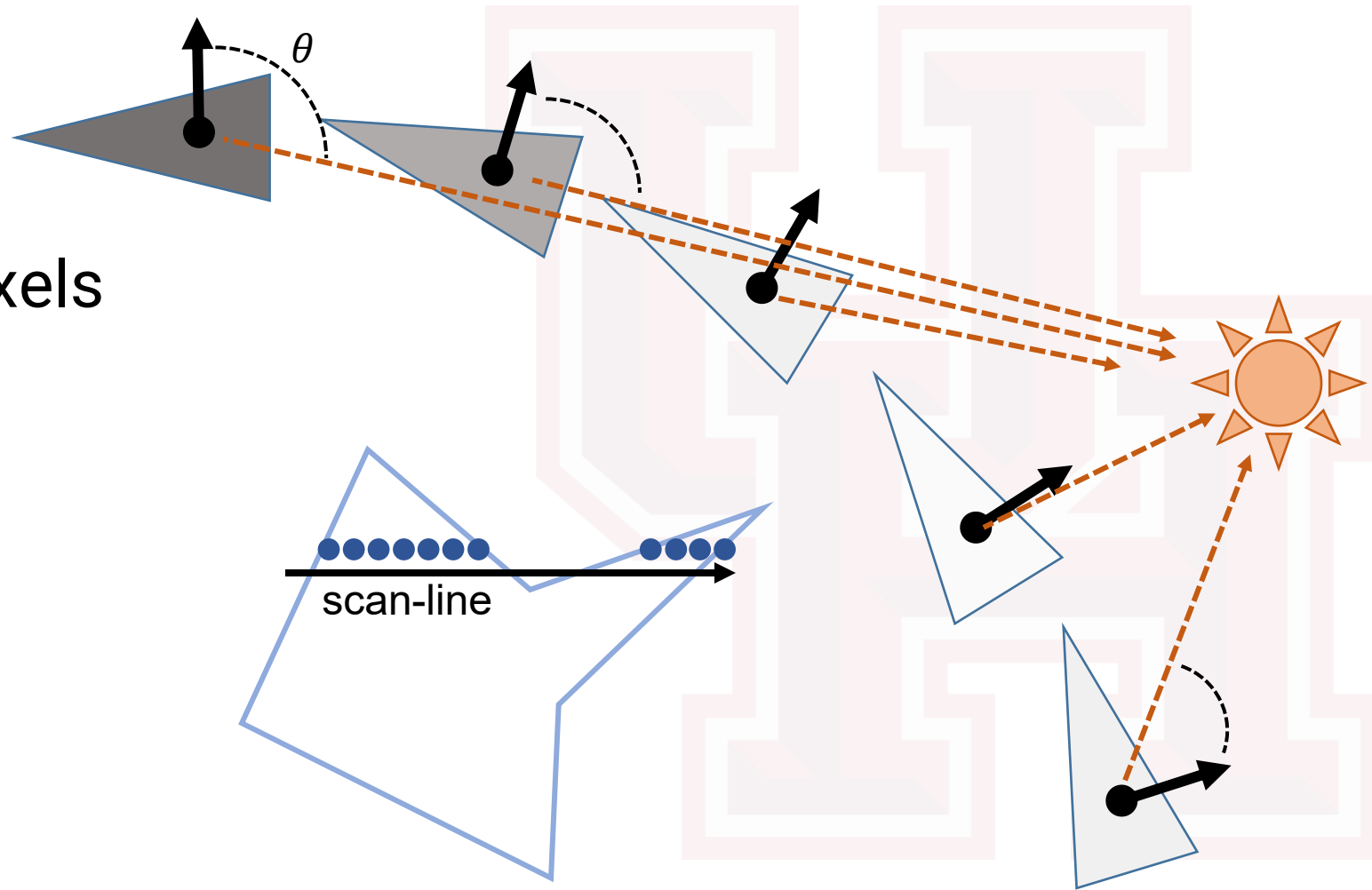
} most of the work



Star Wars (Atari 1983), arcade-museum.com

Shading (1980s)

- Calculate a color value $c = [c_R \ c_G \ c_B]$ for each point inside the polygon
- Scan-line algorithm to fill pixels
- Calculate illumination:
$$\hat{c} = c (n \cdot l)$$
 - n is the surface normal
 - l is the direction to the light source



Shading

```
//calculate the view matrix R
//calculate the projection matrix P
v = vertex[N][3]    //all vertices
n = vector[N]       //all normals
l = vertex[L]       //all light positions
```

```
for each triangle i
```

```
    t[0] = P * R * v[t][0]
    t[1] = P * R * v[t][1]
    t[2] = P * R * v[t][2]
    c = get_color(t)
```

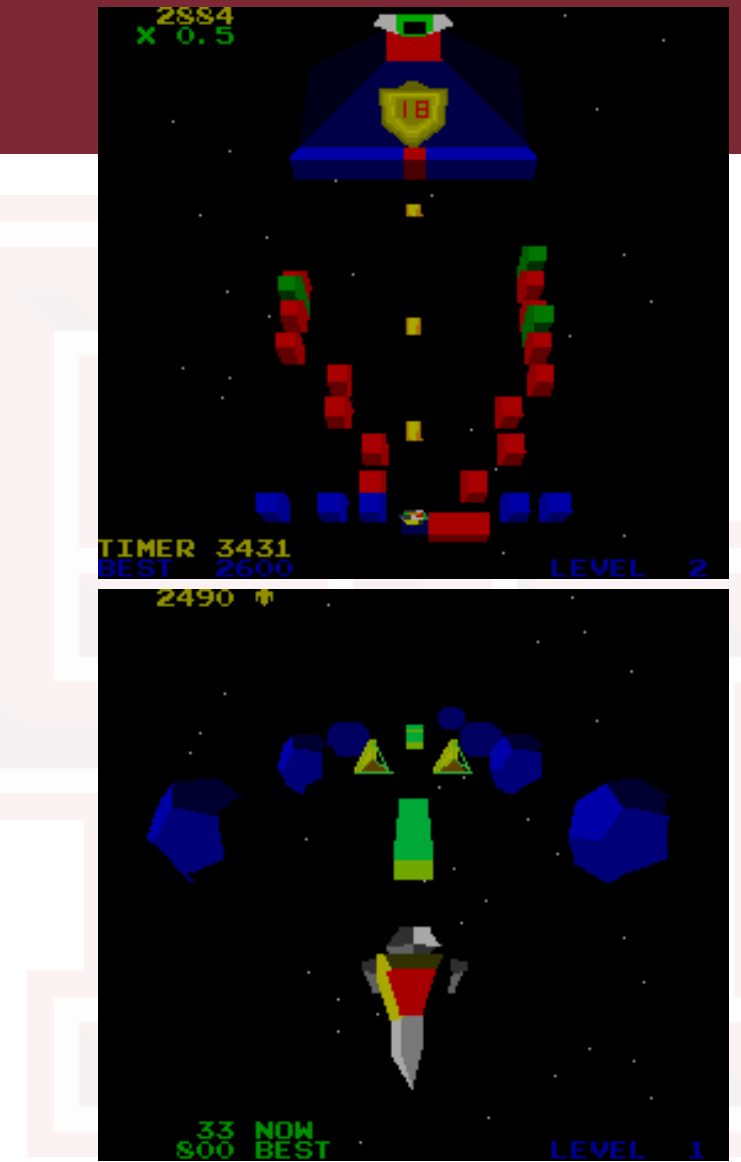
} vertex processing

```
for each pixel p in t
```

```
    for each light j
```

```
         $l_{sum} = l_{sum} + \cos(n \cdot l[j])$ 
        draw_pixel(p,  $l_{sum}$ )
```

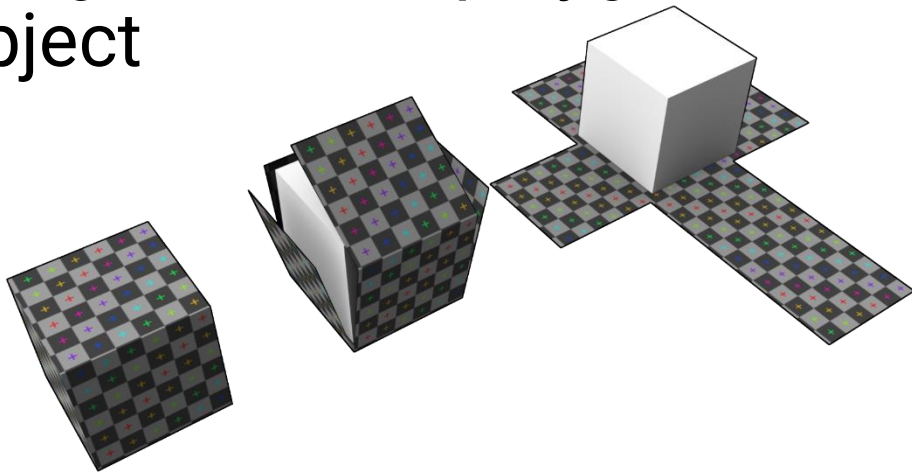
} pixel processing



I, Robot (Atari 1983), arcade-museum.com

Texture Mapping (1990s)

- *texture mapping*: mapping a 2D image onto the polygons of a 3D object



- provides additional color and shading details w/o extra vertices



Voodoo2, Quake, 1996



GeForce2 Ti, Grove, 2000



GeForce2 MX, Creature, 2001



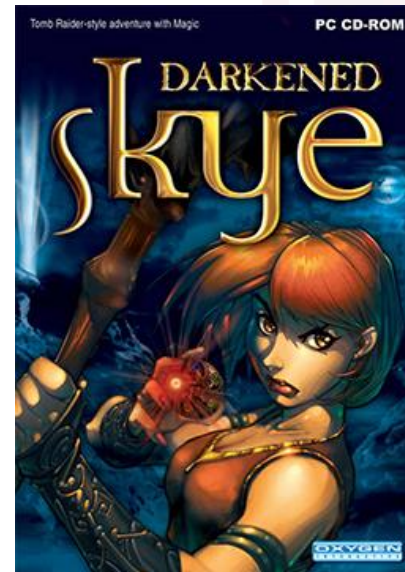
GeForce2 Go, Toy Soldier, 2002

Texture Mapping

```
//calculate the view matrix R
//calculate the projection matrix P
v = vertex[N][3]    //all vertices
n = vector[N]       //all normals
l = vertex[L]       //all light positions
for each triangle i
    t[0] = P * R * v[t][0]
    t[1] = P * R * v[t][1]
    t[2] = P * R * v[t][2]
    for each pixel p in t
        c = get_texture(p)
        for each light j
            l_sum = l_sum + cos(n · l[j])
        draw_pixel(c, l_sum)
```

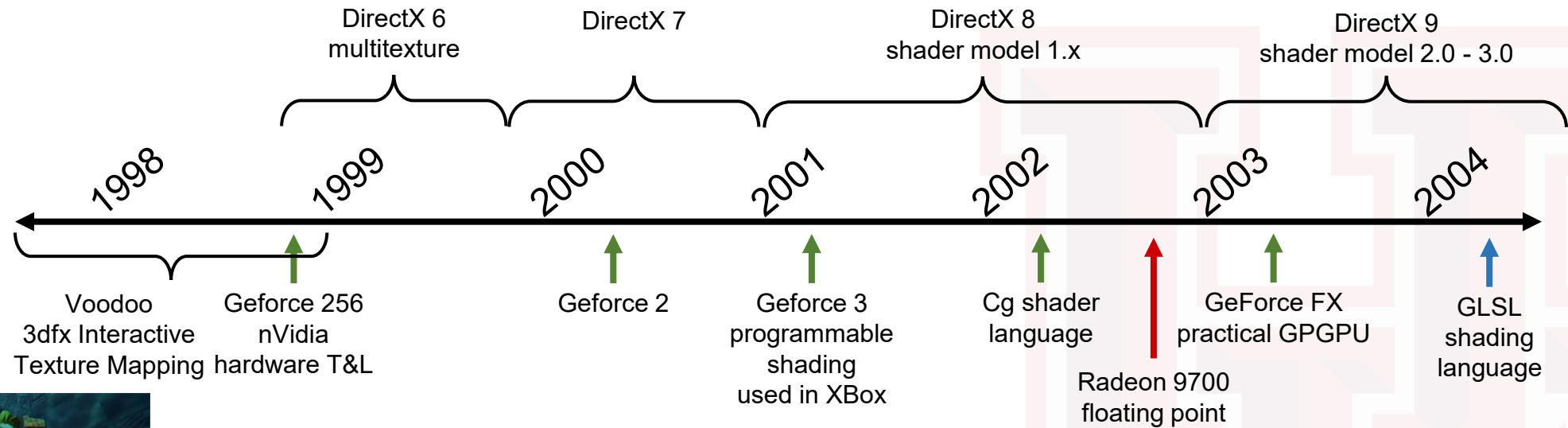
} vertex processing

} fragment processing



Darkened Skye (Boston Animation, 2002)

Programmable Graphics Hardware



1998, Ocarina of Time



1999, System Shock II



2000, Boulder's Gate II



2002, Warcraft III

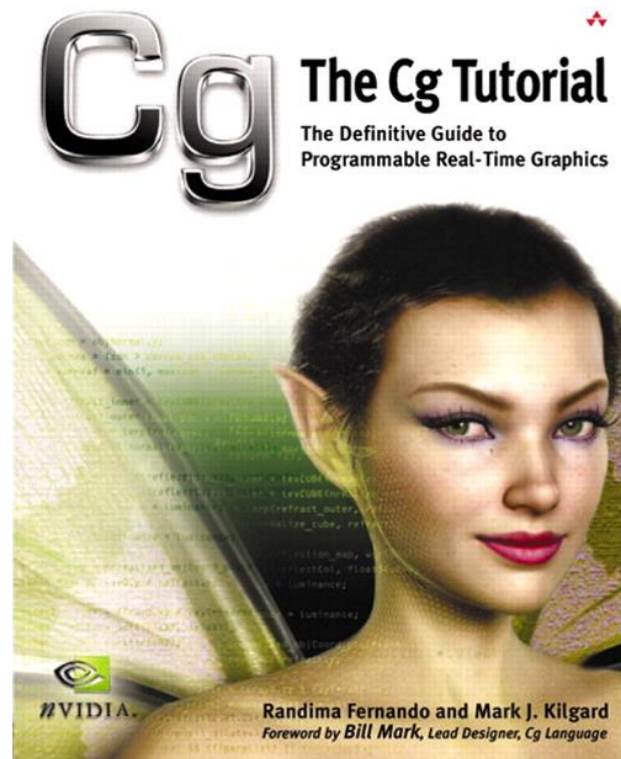


2004, Fable

Cg (C for Graphics)

- GeForce 3 – first programmable shader model (assembly)
- GeForce FX – first practical card for GPGPU
 - floating point precision
 - Cg – C for Computer Graphics

```
Output varying(  
    float2 position : POSITION,  
    float4 color : COLOR,  
    float2 texCoord : TEXCOORD0) {  
    Output OUT;  
    OUT.position = float4(position, 0, 1);  
    OUT.color = color;  
    OUT.texCoord = texCoord;  
    return OUT;  
}
```



GeForce 3 Ti, Chameleon, 2001



GeForce FX 5200, Dawn, 2003

Programming with Shaders

- *general purpose GPU programming* (GPGPU): using the shader framework to do general calculations
- Calculate $C = A + B$ where A and B are large matrices
 - 1) Copy A and B into main memory
 - 2) Send A and B to the GPU as texture maps
 - 3) Render a square to the framebuffer (screen memory), assigning both textures as a “material”
 - 4) Write a simple fragment shader (executed for each pixel):
 - 1) calculate the current pixel position in the matrix
 - 2) retrieve the values for A and B using texture fetches
 - 3) add the values
 - 4) output the sum as a “color”
 - 5) Read the data from the framebuffer to main memory
 - 6) Use the result!

GPGPU – initial challenges and investments

- Debugging was extremely difficult – scale values so that they could be displayed on the screen
- Texture coordinates go from $[0\ 1]$, so you had to map this to your arrays
 - what's the texture coordinate for (123, 67) in a 1024×1024 matrix?
- You could get a PhD implementing reasonably complex algorithms using shading languages and rendering rectangles
- Financial companies would hire game programmers to do market analysis

CUDA (Compute Unified Device Architecture)

- GeForce 8800 (2006)
 - Tesla architecture
 - first CUDA compatible card
- CUDA 1.0 SDK (2007)
 - 8800+ CUDA compatible
 - evolving compute capability (current = 12.1)
- Parallelism model still based on GPUs
 - data parallel
 - single-instruction multiple data (similar to shader programs)
 - Commonly used in graphics applications



GeForce 8800, Froggy, 2006



GeForce GTX Titan, CUDA, Digital Ira, 2013

GeForce Tech Demos



Cascades, GeForce 8 (2007)



Unigine Heaven, GTX 400 (2010)



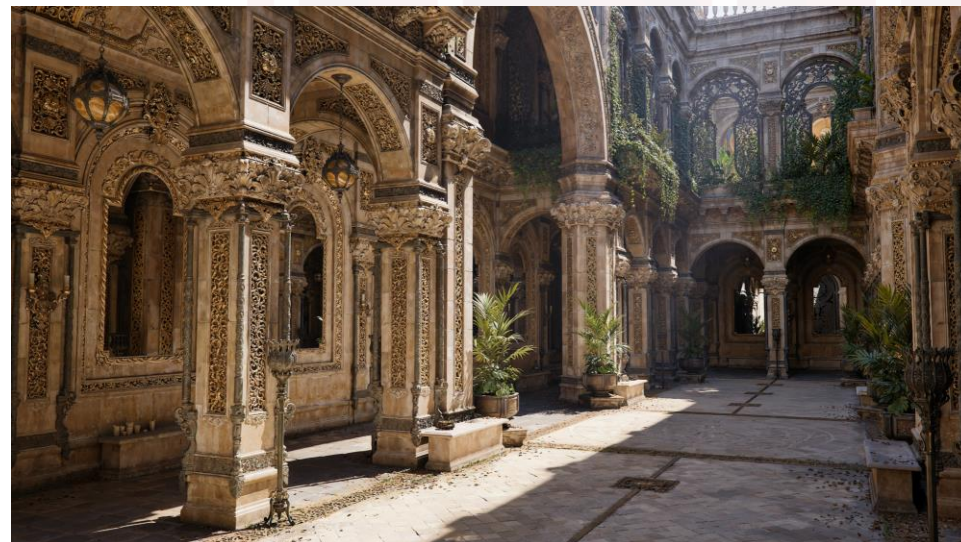
"Ira", GTX TITAN (2013)



Lunar Lander, GTX 900 (2015)



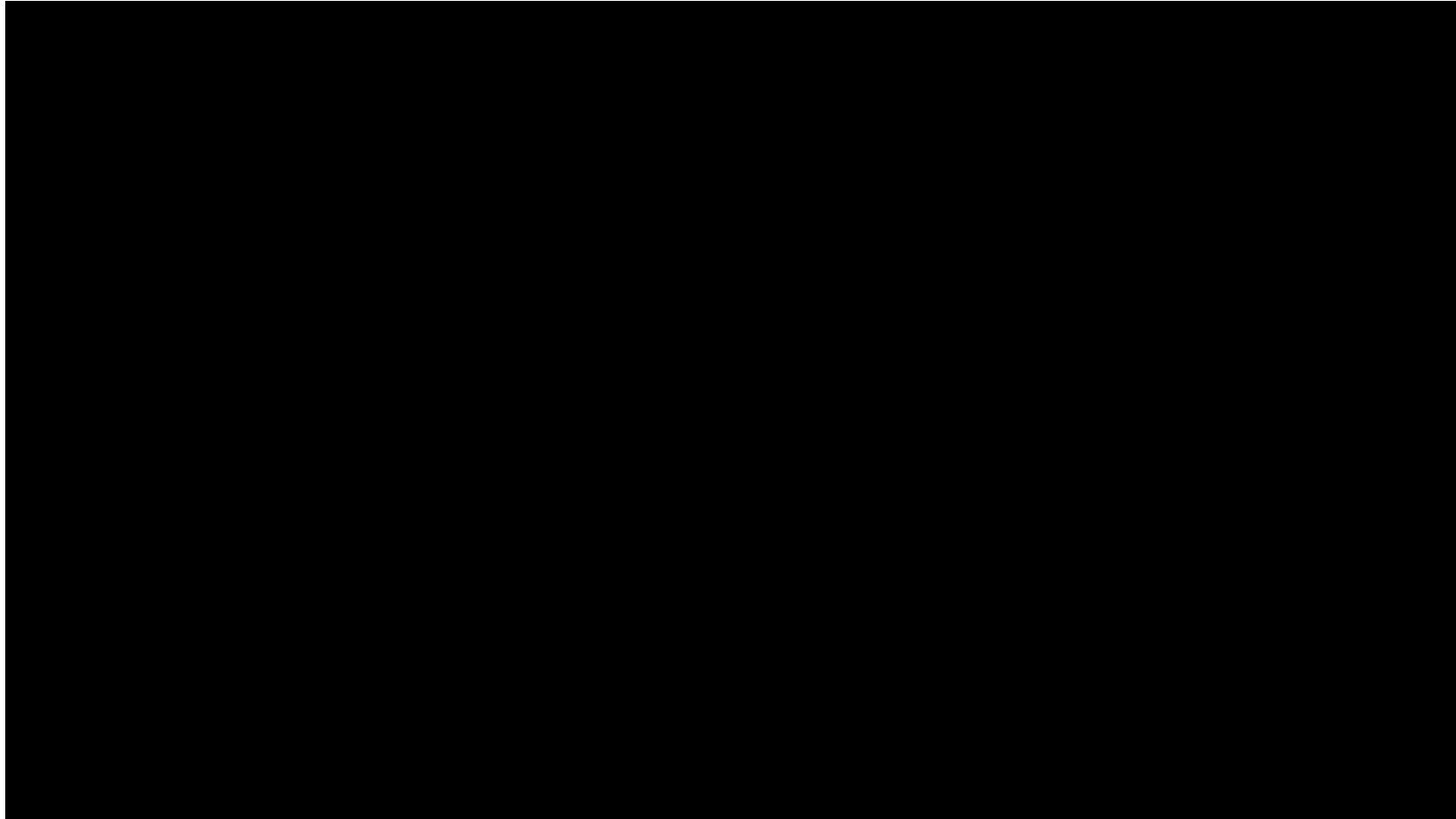
Marbles at Night, RTX Omniverse (2021)



Zorah, RTX 50 (2025)

Modern NVIDIA GPUs

- CUDA Cores:
 - 32-element SIMD vector units
 - hyperthreading
- Tensor Cores:
 - optimized matrix multiply-and-accumulate (MMA)
 - mixed-precision arithmetic (ex. FP16, FP8)
- Ray Tracing Cores:
 - bounding volume hierarchy (BVH) acceleration
 - optimized intersection detection
- GeForce RTX 5080
 - 10752 CUDA Cores (shading units)
 - 336 Tensor Cores (1801 TOPS)
 - 84 Ray Tracing Cores (171 TFLOPS)



Accessing Features

- Graphics APIs
 - OpenGL or Vulkan
 - DirectX
- Shading Languages
 - High-Level Shading Language (HLSL), DirectX
 - OpenGL Shading Language (GLSL)
 - Slang, target-specific compiling
- CUDA API

